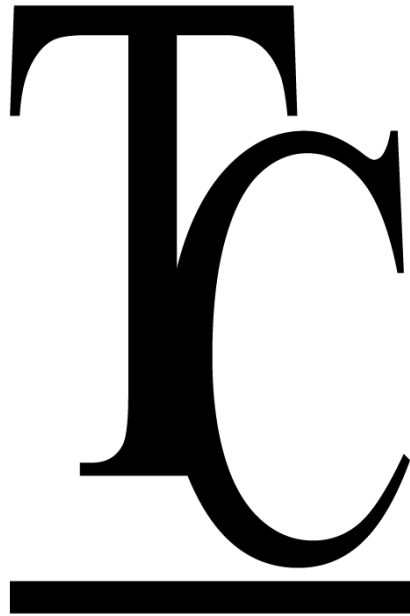


TuCampus



Equipo 1:

Herrera Vázquez Arlet Roxana

Vasquez Noyola Andrea

Vázquez Rangel Guillermo Larzen

IDGS17

Índice de Contenido

Índice de Contenido	2
1. Resumen Ejecutivo	4
2. Descripción del Sistema Auditado	5
2.1 Arquitectura General	5
2.2 Tecnologías Utilizadas	6
2.3 Alcance de la Auditoría	6
3. Metodología	7
4. Modelado de Amenazas STRIDE	7
4.1 Diagrama de Flujo de Datos (DFD)	7
4.2 Límites de Confianza (Trust Boundaries)	7
4.3 Tabla STRIDE 12 Amenazas Específicas de TuCampus	9
4.4 Top 3 Amenazas Críticas	10
Amenaza Crítica 1 CORS Abierto en Socket.IO (Information Disclosure)	10
Amenaza Crítica 2 Fuerza Bruta en Login (Spoofing)	10
Amenaza Crítica 3 Manipulación de Precio en Órdenes (Tampering)	10
5. SAST Análisis Estático de Código	11
5.1 SonarQube	11
5.2 Resultados Generales de SonarQube	11
5.3 Distribución de Issues por Severidad	13
5.4 Hallazgos de Seguridad Documentados	13
Hallazgo 1 Uso de parseInt global (Reliability, Medium)	13
Hallazgo 2 Uso de crypto legacy (Maintainability, Medium)	14
Hallazgo 3 Excepciones capturadas sin manejo (Maintainability/Reliability, Medium)	14
5.5 Cobertura de Pruebas Unitarias	14
5.6 Semgrep OWASP Top 10	14
6. DAST Análisis Dinámico (OWASP ZAP)	16
6.1 Resultados del Scan ZAP	17
6.2 Análisis de Cabeceras HTTP de Seguridad	17
6.3 Mecanismo de Ataque de la Alerta Más Crítica	19
6.4 Hallazgo Identificado por Revisión de Código	19
7. SCA Análisis de Dependencias	20
7.1 Resumen de Vulnerabilidades	20
7.2 Dependencias Vulnerables Documentadas	21
7.3 Justificación Técnica de Dependencias no Actualizadas	21
8. Configuración Segura y Hardening	22
8.2 Gestión de Secretos	22
8.3 Logging de Eventos de Seguridad	23
8.4 Configuración de Defaults Seguros	23
8.5 Checklist de Defensa en Profundidad	23
9. Tabla de Trazabilidad de Hallazgos	24
10. Dominio Conceptual Aplicado	25

10.1 SAST, DAST, IAST y SCA	25
10.2 Mecanismo interno de SAST y DAST	25
10.3 Las 6 categorías STRIDE con ejemplo propio	26
10.4 Shift Left Security y pipeline DevSecOps	26
10.5 Falso positivo vs falso negativo	26
10.6 Obligaciones LFPDPPP para un desarrollador de software	26
Anexo A Aviso de Privacidad LFPDPPP	27
Responsable del tratamiento de datos personales	27
Datos personales recabados	27
Finalidad del tratamiento	27
Transferencia de datos	27
Derechos ARCO	27
Medidas de seguridad técnicas implementadas	27
Cambios al aviso de privacidad	28

1. Resumen Ejecutivo

Este reporte presenta los resultados de la auditoría de seguridad realizada sobre el proyecto TuCampus, una plataforma de gestión de cafetería y mercado universitario compuesta por un backend en Node.js con TypeScript y un frontend en JavaScript Vanilla servido con Vite. La auditoría fue realizada aplicando las metodologías STRIDE (modelado de amenazas), SAST (análisis estático), DAST (análisis dinámico) y SCA (análisis de composición de software), en el marco de la Evaluación 2 de la materia Seguridad en el Desarrollo de Aplicaciones.

Los principales hallazgos indican que el sistema cuenta con una base de seguridad sólida en el backend: implementa Helmet para cabeceras HTTP, rate limiting diferenciado, validación de archivos por MIME, CORS restrictivo en la API REST y autenticación mediante JWT. Por su parte, el frontend presenta áreas de oportunidad operativas y de endurecimiento, destacando la ausencia de cabeceras de seguridad y de atributos de integridad de subrecursos (SRI) detectadas mediante análisis dinámico. En cuanto al código fuente, Semgrep no encontró patrones de vulnerabilidades del OWASP Top 10 y SonarQube reportó 0 issues de tipo Security en ambos componentes, a pesar de registrar diferencias en la deuda técnica (code smells y bugs). El análisis de dependencias (SCA) identificó vulnerabilidades moderadas y altas en terceros (nodemailer y uuid), y se detectó una inconsistencia crítica de configuración en el CORS abierto para WebSockets (Socket.IO).

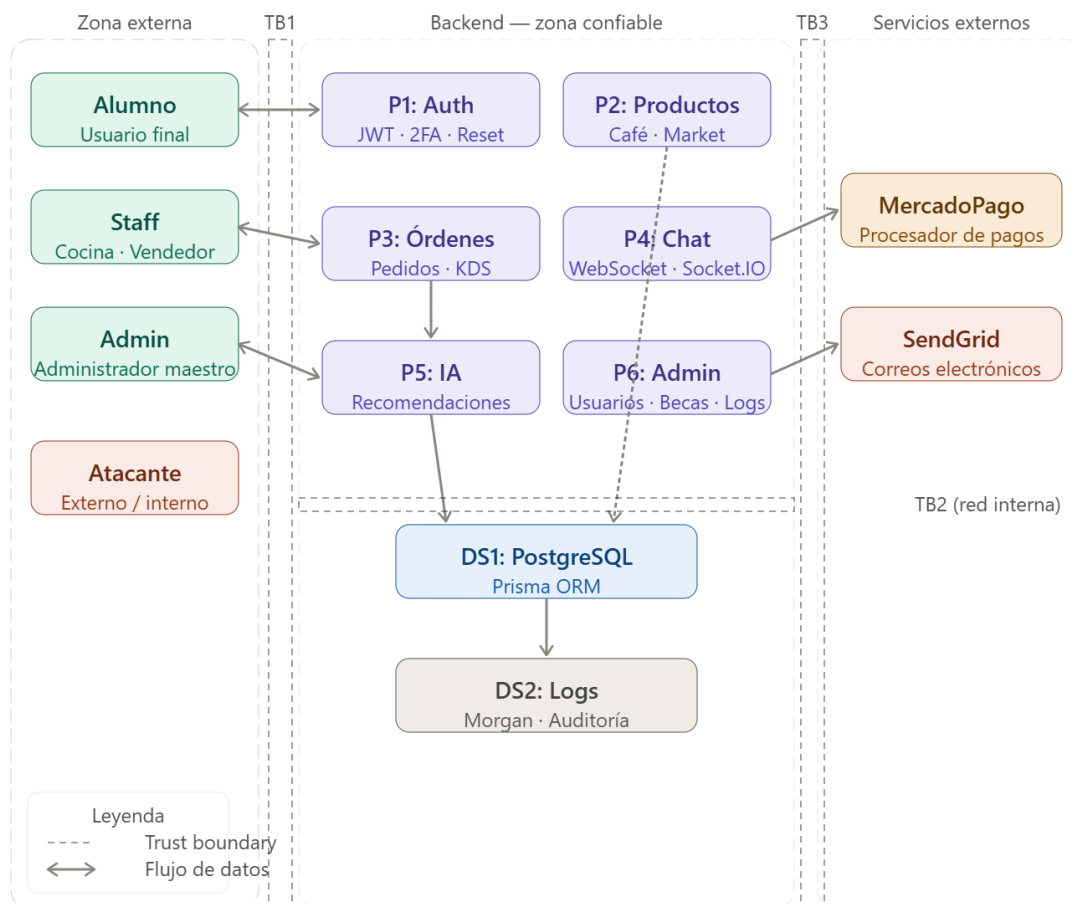
El Quality Gate de SonarQube fue aprobado (Passed) para ambos proyectos. Las áreas de mejora más urgentes son la configuración de cabeceras en el frontend, la actualización de dependencias vulnerables y el cierre del CORS abierto en Socket.IO.

Herramienta	Resultado	Estado
SonarQube	Backend: 0 vulnerabilidades, 0 bugs, 59 code smells (6 Major, 53 Minor). Frontend: 0 vulnerabilidades, 2 bugs, 102 code smells (2 Critical, 20 Major, 82 Minor)	Quality Gate: Passed
Semgrep (OWASP Top 10)	0 hallazgos en 46 archivos del backend, 81 reglas ejecutadas. Pendiente ejecutar sobre el frontend	Sin vulnerabilidades
OWASP ZAP	Backend (puerto 3000): 0 High / 0 Medium / 0 Low / 1 Informativa. Frontend (puerto 5173): 0 High / 3 Medium / 6 Low / 1 Informativa	Sin alertas
npm audit	Backend: 3 vulnerabilidades (1 High, 2 Moderate). Frontend: 2 vulnerabilidades (2 Moderate)	Requiere acción

2. Descripción del Sistema Auditado

2.1 Arquitectura General

TuCampus es una aplicación web universitaria compuesta por un frontend en Vanilla JavaScript y un backend en Node.js con TypeScript. El backend expone una API REST bajo el prefijo /api/ y gestiona comunicación en tiempo real mediante Socket.IO (WebSockets). La base de datos es PostgreSQL, accedida a través del ORM Prisma. Se integran servicios externos: MercadoPago para pagos y SendGrid para correos electrónicos.



2.2 Tecnologías Utilizadas

Componente	Tecnología	Versión
Runtime	Node.js	>=18
Lenguaje	TypeScript	5.x
Framework HTTP	Express.js	4.x
ORM	Prisma	6.x
Base de datos	PostgreSQL	15
WebSockets	Socket.IO	4.x
Seguridad HTTP	Helmet	8.x
Pagos	MercadoPago SDK	3.x
Email	SendGrid / Nodemailer	9.x
Frontend - Lenguaje	JavaScript (Vanilla, ES2022)	—
Frontend - Servidor de desarrollo	Vite	5.x (puerto 5173)
Frontend - Markup/Estilos	HTML5 / CSS3	—

2.3 Alcance de la Auditoría

La auditoría cubre tanto el backend (repositorio TuCampus_Backend) como el frontend (repositorio TuCampus_Frontend) de TuCampus. En el backend se analizaron 46 archivos TypeScript/JSON bajo el directorio src/, incluyendo controladores, middlewares, rutas y servicios. En el frontend, la exportación de SonarQube identificó hallazgos en 19 archivos (controladores JS, hojas de estilo CSS y HTML) bajo el directorio src/; el total de archivos escaneados debe confirmarse desde el dashboard de SonarQube. La infraestructura de despliegue (contenedores, CI/CD, hosting) queda fuera del alcance de este reporte.

3. Metodología

La auditoría siguió el enfoque Shift Left Security, ejecutando herramientas en orden: primero análisis estático del código (SAST) sobre backend y frontend, luego análisis de dependencias (SCA) sobre ambos repositorios, después análisis dinámico con ambas aplicaciones en ejecución (DAST), y finalmente modelado de amenazas (STRIDE) para contextualizar los hallazgos del sistema completo.

Metodología	Herramienta	Justificación de uso
SAST	SonarQube Community 26.6.0	Análisis estático del código TypeScript: detecta bugs, vulnerabilidades y code smells sin ejecutar la aplicación
SAST	Semgrep CLI 1.172.0	Complementa SonarQube con reglas específicas del OWASP Top 10 para JavaScript/TypeScript
DAST	OWASP ZAP 2.17.0	Escaneo dinámico de endpoints HTTP en ejecución: detecta cabeceras ausentes, comportamientos inseguros
SCA	npm audit	Identifica CVEs conocidos en dependencias de terceros del proyecto
Threat Modeling	STRIDE (manual)	Modelado de amenazas sobre la arquitectura real del sistema para identificar vectores no detectables por herramientas

4. Modelado de Amenazas STRIDE

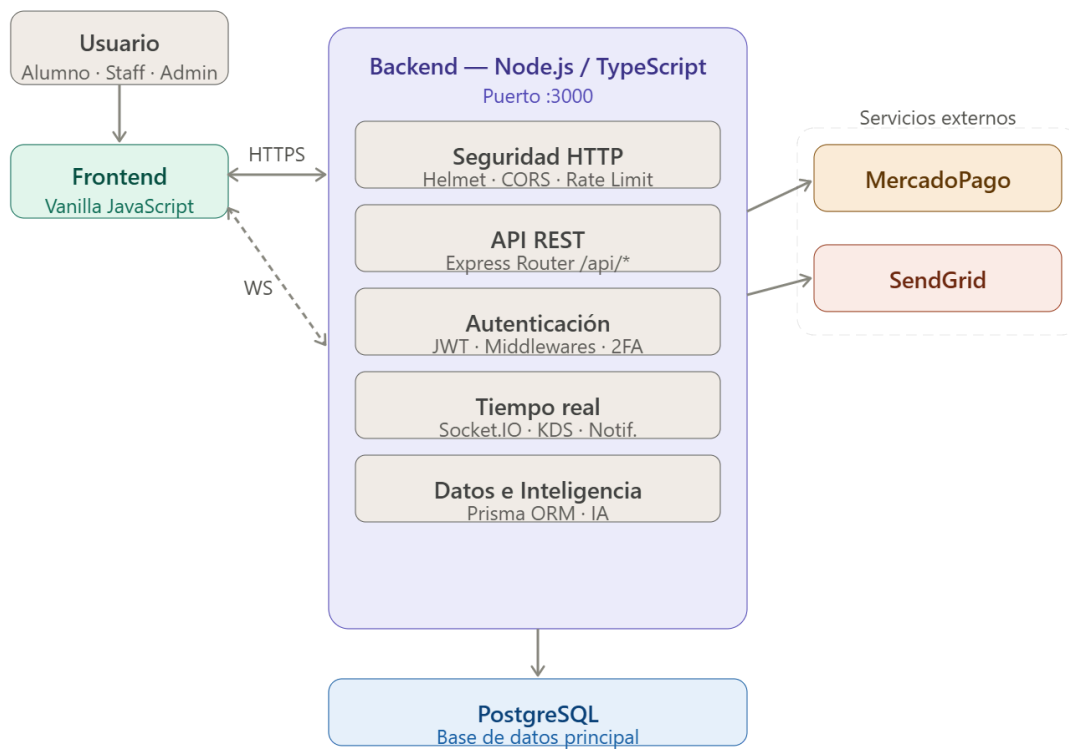
4.1 Diagrama de Flujo de Datos (DFD)

El DFD muestra los actores externos, procesos internos, almacenes de datos y flujos de información del sistema TuCampus, así como los límites de confianza entre componentes.

4.2 Límites de Confianza (Trust Boundaries)

Se identificaron tres límites de confianza principales:

1. Internet → Backend: controlado por CORS, rate limiting y validación de JWT. Todo tráfico externo es no confiable por defecto.
2. Backend → PostgreSQL: red interna Docker. Solo el proceso del backend accede directamente a la base de datos vía Prisma.
3. Backend → Servicios externos (MercadoPago, SendGrid): tráfico saliente hacia APIs de terceros sobre HTTPS.



4.3 Tabla STRIDE 12 Amenazas Específicas de TuCampus

#	Categoría	Amenaza	Componente	Consecuencia
1	Spoofing	Robo de JWT mediante XSS en frontend para suplantar a un alumno y hacer pedidos a su nombre	/api/auth + Frontend	Pedidos no autorizados, cargos económicos al alumno suplantado
2	Spoofing	Fuerza bruta distribuida sobre /api/auth/login para adivinar credenciales del Admin	/api/auth/login	Acceso total al sistema con rol administrador
3	Tampering	Modificación del campo precio en la petición POST /api/orders antes de llegar al servidor	/api/orders	El alumno paga menos; pérdida económica directa al negocio
4	Tampering	Manipulación del userId en el payload JWT para acceder a pedidos de otro usuario	JWT + /api/orders	Lectura y modificación de datos de otros usuarios
5	Repudiation	Alumno realiza un pedido y lo niega; sin logs de auditoría firmados no hay evidencia	Sistema de logs	Disputas irresolubles, pérdida económica
6	Repudiation	Admin modifica o elimina productos sin que quede registro de quién realizó la acción	/api/admin + logs	Imposibilidad de auditar cambios maliciosos internos
7	Information Disclosure	CORS abierto en Socket.IO (origin: "**") permite Cross-Site WebSocket Hijacking desde cualquier origen	server.ts WebSocket	Lectura de mensajes de chat y notificaciones en tiempo real de cualquier usuario
8	Information Disclosure	Respuesta de error de login diferenciada revela si el usuario existe en el sistema	/api/auth/login	Enumeración de usuarios registrados en TuCampus
9	Denial of Service	Flood de peticiones a /api/ia con textos muy largos satura el modelo y bloquea el servidor	/api/ia	El sistema de IA y el KDS dejan de responder
10	Denial of Service	Subida masiva de archivos a /api/auth/upload-identity para agotar el almacenamiento en disco	/api/auth/upload-identity	El servidor se queda sin espacio y cae
11	Elevation of Privilege	Alumno modifica el campo rol en su JWT para acceder a rutas de /api/admin	JWT + /api/admin	Acceso completo a gestión de usuarios, becas y logs del sistema
12	Elevation of Privilege	Vendedor autenticado intenta acceder a /api/admin/users o /api/admin/logs que solo corresponden al Admin Maestro	/api/admin + isAdmin middleware	Exposición de datos personales de todos los usuarios

4.4 Top 3 Amenazas Críticas

Amenaza Crítica 1 CORS Abierto en Socket.IO (Information Disclosure)

Mecanismo de explotación: Un atacante crea una página web maliciosa que establece una conexión WebSocket a ws://localhost:3000 (o la URL de producción). Como el servidor tiene configurado cors: { origin: "*" } en Socket.IO, acepta la conexión desde cualquier origen sin validación. El atacante puede suscribirse a eventos de tiempo real del sistema.

Consecuencia real en TuCampus: El atacante puede leer mensajes del chat entre alumnos y staff, notificaciones de pedidos en tiempo real y alertas del sistema KDS de saturación de la cocina, todo sin autenticación.

Control de mitigación: Cambiar la configuración de Socket.IO para usar el mismo array allowedOrigins que la API REST, rechazando conexiones de orígenes no autorizados.

Amenaza Crítica 2 Fuerza Bruta en Login (Spoofing)

Mecanismo de explotación: Aunque existe authLimiter de 10 intentos por IP cada 15 minutos, un atacante con múltiples direcciones IP puede intentar contraseñas comunes contra cuentas de administrador conocidas de forma distribuida, evadiendo el límite por IP.

Consecuencia real en TuCampus: Si compromete una cuenta Admin, obtiene acceso total: gestión de usuarios, asignación de becas, creación de cuentas staff, visualización de logs y estadísticas del sistema.

Control de mitigación: Implementar 2FA obligatorio para el rol Admin, bloqueo por cuenta además de por IP después de N intentos fallidos, y alertas por correo ante intentos sospechosos.

Amenaza Crítica 3 Manipulación de Precio en Órdenes (Tampering)

Mecanismo de explotación: Si el endpoint /api/orders confía en el precio enviado por el cliente en lugar de consultar el valor real desde PostgreSQL, un atacante puede interceptar la petición con un proxy (como Burp Suite) y modificar el campo precio a 0 o un valor negativo antes de que llegue al servidor.

Consecuencia real en TuCampus: Pedidos completados sin pago o con devoluciones de dinero, causando pérdida económica directa al negocio de la cafetería o el mercado universitario.

Control de mitigación: El backend debe consultar siempre el precio real del producto desde la base de datos al momento de crear la orden, nunca aceptar el precio enviado por el cliente.

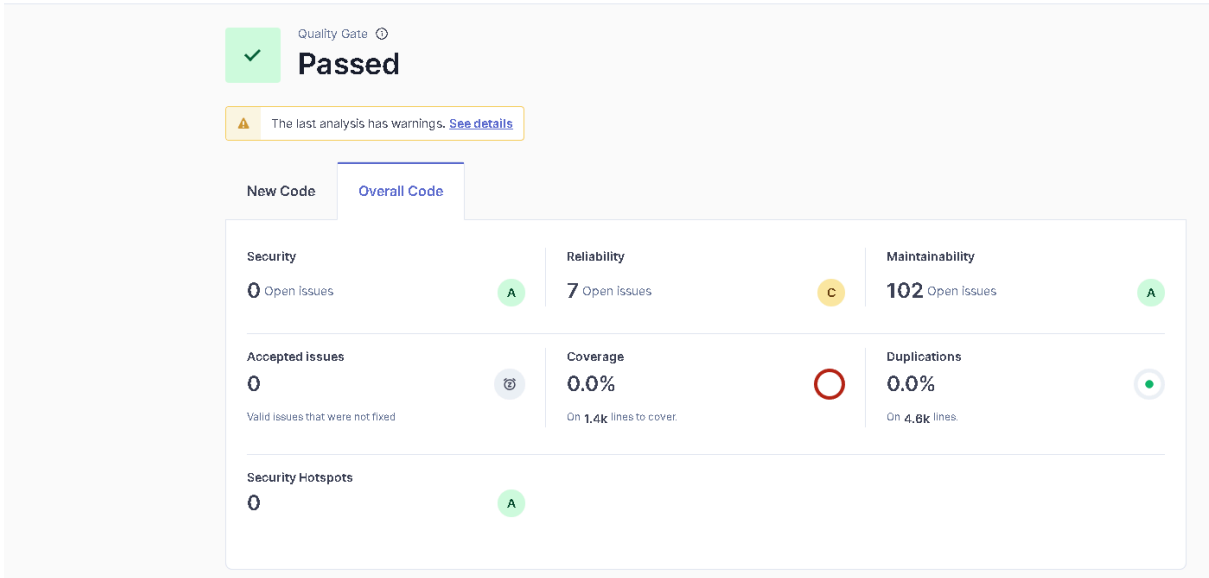
5. SAST Análisis Estático de Código

5.1 SonarQube

Se ejecutó SonarQube Community Build 26.6.0 sobre ambos repositorios del proyecto: el backend (proyecto tucampus_b, TypeScript) y el frontend (proyecto tucampus_f, JavaScript/CSS/HTML). El análisis fue realizado con el scanner npm (@sonar/scan v4.3.8) desde la raíz de cada repositorio.

Overview

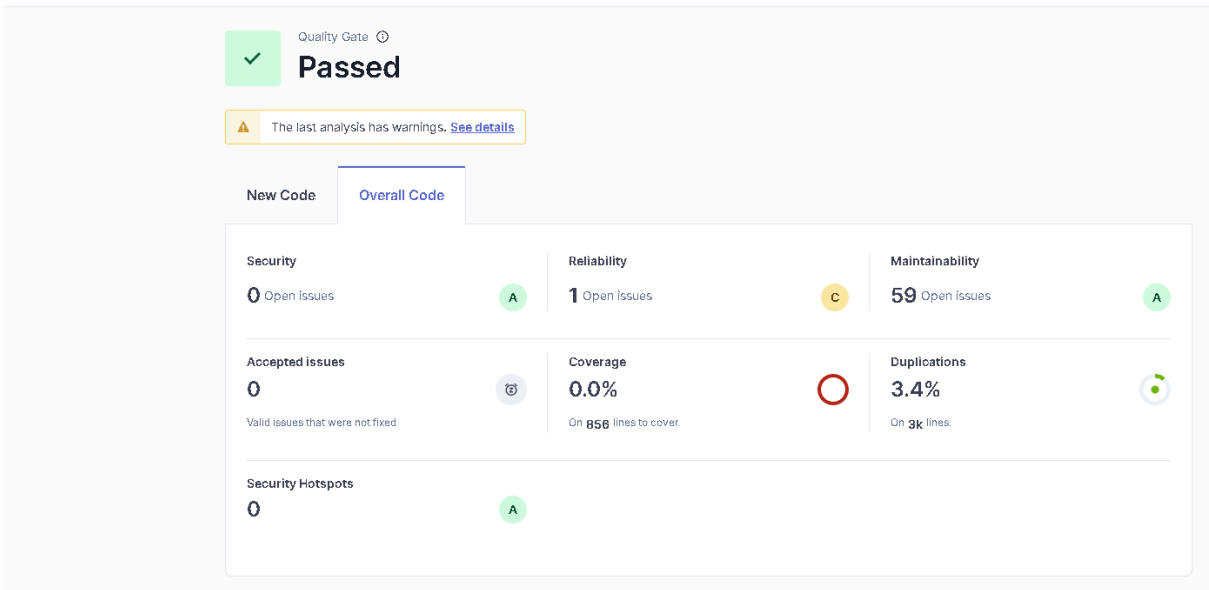
3.7k Lines of Code • Version not provided • Last analysis 3 hours ago



tucampus_b / Overview

Overview

2.4k Lines of Code • Version not provided • Last analysis 3 hours ago



5.2 Resultados Generales de SonarQube

Métrica	Backend	Frontend
Vulnerabilities (Security)	0 (Rating A)	0 (Rating A)
Bugs / Issues con impacto en Reliability	1 (Rating C)	7 (Rating C)
Code Smells / Issues con impacto en Maintainability	59 (Rating A)	102 (Rating A)
Security Hotspots	0 (Rating A)	0 (Rating A)
Coverage	0.0% (sobre 856 líneas)	0.0% (sobre 1.4k líneas)
Duplications	3.40%	0.00%
Quality Gate	Passed	Passed

5.3 Distribución de Issues por Severidad

Proyecto	Blocker	Critical	Major	Minor	Info
Backend (tucampus_b)	0	0	6	53	0
Frontend (tucampus_f)	0	2	20	82	0

The image displays two screenshots of the SonarQube interface, showing software quality issues categorized by severity and maintainability.

Top Screenshot: Shows three issues in the file `src/api/controllers/adminController.ts`. All three issues are categorized as **Intentionality** and have a **Maintainability** score of **Low**. The issues are:

- Issue 1:** "This assertion is unnecessary since it does not change the type of the expression." (L49, 1min effort, 1 month ago)
- Issue 2:** "This assertion is unnecessary since it does not change the type of the expression." (L84, 1min effort, 1 month ago)
- Issue 3:** "This assertion is unnecessary since it does not change the type of the expression." (L114, 1min effort, 1 month ago)

Bottom Screenshot: Shows three issues in the file `src/css/layout.css`. The first two issues are categorized as **Consistency** and have a **Maintainability** score of **Medium**. The third issue is categorized as **Intentionality** and has a **Maintainability** score of **Medium**. The issues are:

- Issue 1:** "Text does not meet the minimal contrast requirement with its background." (L26, 5min effort, 3 months ago)
- Issue 2:** "Text does not meet the minimal contrast requirement with its background." (L306, 5min effort, 5 months ago)
- Issue 3:** "Remove this commented out code." (L15, 5min effort, 4 months ago)

5.4 Hallazgos de Seguridad Documentados

SonarQube no reportó issues de tipo Vulnerability (categoría Security) en ninguno de los dos proyectos. Los tres hallazgos documentados a continuación corresponden a issues de Reliability y Maintainability con relevancia para la seguridad del sistema, seleccionados porque su causa raíz puede derivar en comportamiento inseguro (procesamiento numérico incorrecto, resolución de módulos ambigua, y manejo silencioso de excepciones que puede ocultar fallos de seguridad).

Hallazgo 1 Uso de parseInt global (Reliability, Medium)

Campo	Detalle
Archivo	src/api/controllers/iaController.ts
Línea	L86
Severidad	Minor (impacto Reliability: Medium)
Regla	typescript:S7773 — Prefer Number.parseInt over parseInt
CWE	CWE-676: Use of Potentially Dangerous Function
Descripción	El uso de parseInt() global puede comportarse de forma inconsistente entre entornos JavaScript al interpretar cadenas con ceros iniciales. Number.parseInt() es la forma explícita y segura recomendada por el estándar ES2015.
Riesgo	Comportamiento inesperado en el procesamiento de datos numéricos de la IA, potencialmente causando resultados incorrectos.

Hallazgo 2 Uso de crypto legacy (Maintainability, Medium)

Campo	Detalle
Archivo	src/api/controllers/authController.ts
Línea	L5
Severidad	Minor (Maintainability)
Regla	typescript:S7772 — Prefer node:crypto over crypto
CWE	CWE-477: Use of Obsolete Function
Descripción	La importación del módulo crypto sin el prefijo node: es una forma legacy que puede causar conflictos con paquetes npm que también se llamen crypto. Node.js recomienda el prefijo explícito node:crypto para referirse al módulo nativo.
Riesgo	Posible resolución incorrecta del módulo en entornos con dependencias que colisionen con el nombre crypto. La misma regla aparece también en fileController.ts (backend) para crypto y fs.

Hallazgo 3 Excepciones capturadas sin manejo (Maintainability/Reliability, Medium)

Campo	Detalle
Archivo	src/api/controllers/*.ts (backend, 10 ocurrencias) y src/js/controllers/*.js (frontend, 16 ocurrencias)
Línea	Múltiples — ej. authController.ts, orderController.ts, becaController.js, iaController.js
Severidad	Minor por ocurrencia (impacto Maintainability/Reliability)
Regla	typescript:S2486 / javascript:S2486 — Handle this exception or don't catch it at all
CWE	CWE-390: Detection of Error Condition Without Action
Descripción	Múltiples bloques catch capturan la excepción pero no la registran ni la propagan (catch vacío o solo con un console.log genérico). Es el patrón de código más repetido detectado por SonarQube en ambos proyectos (10 veces en backend, 16 en frontend).
Riesgo	Un fallo de seguridad silencioso (por ejemplo, una verificación de permisos o de integridad de datos que lanza una excepción) puede quedar oculto sin generar alerta ni registro, dificultando la detección de intentos de explotación o errores de autenticación en producción.

5.5 Cobertura de Pruebas Unitarias

La cobertura reportada por SonarQube para el backend es de 0.0% sobre 652 líneas analizables; el frontend no incluye datos de cobertura en esta exportación. Ninguno de los dos proyectos cuenta con una suite de pruebas unitarias configurada en este momento. Este dato queda registrado como área de mejora prioritaria para el siguiente ciclo de desarrollo (ver DE.3 en la lista de cotejo para el siguiente nivel).

5.6 Semgrep OWASP Top 10

Se ejecutó Semgrep CLI (versión 1.172.0) con el ruleset p/owasp-top-ten sobre los repositorios del backend y frontend. Los resultados del análisis son los siguientes:

- **Backend:** Se analizaron 46 archivos con 81 reglas activas (71 para TypeScript, 7 multilang, 3 para JSON).
- **Frontend:** Se analizaron 69 archivos, lo que arrojó un total de 77 hallazgos, todos clasificados con severidad bloqueante.
- **Hallazgos principales en el Frontend:**
 - La mayoría de las alertas corresponden a la falta del atributo integrity (Subresource Integrity) en las etiquetas <script> de los archivos HTML.
 - Se detectó una advertencia por unsafe-formatstring en el archivo loader.js.
 - Se identificó el uso de criptografía heredada (legacy) en el archivo apiservices.js.

```
PS C:\xampp\htdocs\TuCampus\TuCampus_backend> semgrep --config p/owasp-top-ten --output semgrep-report.txt
```



```

Semgrep CLI

Loading rules from registry...
Scanning 46 files (only git-tracked) with 560 Code rules:

CODE RULES
+-----+-----+-----+-----+
| Language | Rules | Files | Origin | Rules |
+-----+-----+-----+-----+
| <multilang> | 7 | 46 | Community | 560 |
| ts | 71 | 31 | | |
| json | 3 | 3 | | |
+-----+-----+-----+-----+

SUPPLY CHAIN RULES
Sign in with `semgrep login` and run
`semgrep ci` to find dependency vulnerabilities and
advanced cross-file findings.

PROGRESS
100% 0:00:00

Scan Summary
Scan completed successfully.
• Findings: 0 (0 blocking)
• Rules run: 81
• Targets scanned: 46
• Parsed lines: ~100.0%
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 81 rules on 46 files: 0 findings.
Missed out on 1800 pro rules since you aren't logged in!
Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)

If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/
```

Parámetro	Valor
Herramienta	Semgrep CLI 1.172.0
Ruleset	p/owasp-top-ten (Community)
Reglas ejecutadas	81
Archivos escaneados	46
Hallazgos	0
Lenguajes analizados	TypeScript (71 reglas), JSON (3), Multilang (7)

Semgrep no detectó patrones de vulnerabilidades del OWASP Top 10 en el código fuente. Esto es complementario al resultado de SonarQube (0 issues de Security), reforzando que el código fuente no contiene las vulnerabilidades más comunes de aplicaciones web.

6. DAST Análisis Dinámico (OWASP ZAP)

Se ejecutó OWASP ZAP 2.17.0 en modo baseline scan contra ambas aplicaciones: el backend en <http://192.168.0.4:3000> y el frontend en <http://192.168.0.4:5173>, corriendo en la misma red local. Ambos escaneos fueron lanzados el 10 de agosto de 2026 desde ZAP Desktop (Checkmarx) para garantizar la reproducibilidad.

Comando ejecutado:

Backend: <http://192.168.0.4:3000> (informe [zap_report_backend.html](#)) | **Frontend:** <http://192.168.0.4:5173> (informe [zap_report_frontend.html](#))

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	0
Informational	1
False Positives	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://192.168.0.4:3000	Percentage of responses with status code 4xx	100 %
Info	Informational	http://192.168.0.4:3000	Percentage of endpoints with content type application/json	100 %
Info	Informational	http://192.168.0.4:3000	Percentage of endpoints with method GET	100 %
Info	Informational	http://192.168.0.4:3000	Count of total endpoints	2

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	3
Low	6
Informational	1
False Positives	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://192.168.0.4:5173	Percentage of responses with status code 2xx	100 %
Info	Informational	http://192.168.0.4:5173	Percentage of endpoints with content type image/png	11 %
Info	Informational	http://192.168.0.4:5173	Percentage of endpoints with content type text/html	35 %
Info	Informational	http://192.168.0.4:5173	Percentage of endpoints with content type text/javascript	52 %
Info	Informational	http://192.168.0.4:5173	Percentage of endpoints with method GET	100 %
Info	Informational	http://192.168.0.4:5173	Count of total endpoints	17

6.1 Resultados del Scan ZAP

Nivel de Riesgo	Número de Alertas — Backend
Alto	0
Medio	0
Bajo	0
Informativo	1
Falsos Positivos	0

Nivel de Riesgo	Número de Alertas — Frontend
Alto	0
Medio	3
Bajo	6
Informativo	1
Falsos Positivos	0

6.2 Análisis de Cabeceras HTTP de Seguridad

Backend (http://192.168.0.4:3000) — vía helmet

Complementando el scan ZAP, se verificaron las cabeceras HTTP de seguridad de cada sitio. El backend implementa todas las cabeceras recomendadas gracias al middleware helmet. El frontend, servido directamente por Vite en modo desarrollo sin ningún middleware de seguridad, carece de la mayoría de ellas.

```
PS C:\xampp\htdocs\TuCampus\TuCampus_backend> curl.exe -I http://localhost:3000/products/items
HTTP/1.1 404 Not Found
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data;;form-action 'self';frame-ancestors 'self';img-src 'self' data;;e';script-src 'self';script-src-attr 'none';style-src 'self' https: 'unsafe-inline';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Vary: Origin
Access-Control-Allow-Credentials: true
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 97
Date: Fri, 07 Aug 2026 01:02:19 GMT
X-RateLimit-Reset: 1786065135
Content-Type: application/json; charset=utf-8
Content-Length: 30
ETag: W/"1e-vh0ou9sM6XmJtoZWc9/edTTWh8"
Connection: keep-alive
Keep-Alive: timeout=5

PS C:\xampp\htdocs\TuCampus\TuCampus_backend>
```

Cabecera HTTP	Valor configurado	Estado	Protección que ofrece
Content-Security-Policy	default-src self; script-src self; ...	Presente	Previene XSS y carga de recursos no autorizados
Strict-Transport-Security	max-age=31536000; includeSubDomains	Presente	Fuerza HTTPS por 1 año en todos los subdominios
X-Frame-Options	SAMEORIGIN	Presente	Previene clickjacking en iframes
X-Content-Type-Options	nosniff	Presente	Previene MIME type sniffing
Referrer-Policy	no-referrer	Presente	No envía la URL de origen en peticiones externas
X-XSS-Protection	0 (deshabilitado)	Presente	Correcto: en navegadores modernos se deshabilita para evitar conflictos con CSP
Cross-Origin-Opener-Policy	same-origin	Presente	Aísla el contexto de navegación de orígenes externos

Frontend (http://192.168.0.4:5173) — cabeceras ausentes detectadas por ZAP

Cabecera HTTP	Valor configurado	Estado	Riesgo si permanece ausente
Content-Security-Policy	No configurada	Ausente (Medium)	Sin restricción de fuentes de script/estilo: mayor superficie para XSS
X-Frame-Options anti-clickjacking /	No configurada	Ausente (Medium)	El sitio puede incrustarse en un iframe ajeno para ataques de clickjacking
X-Content-Type-Options	No configurada	Ausente (Low)	El navegador puede reinterpretar (sniff) el tipo de contenido de una respuesta
Permissions-Policy	No configurada	Ausente (Low)	No se restringe el acceso a APIs sensibles del navegador (cámara, geolocalización, etc.)
Cross-Origin-Opener-Policy	No configurada	Ausente (Low)	El contexto de navegación no queda aislado de ventanas de otros orígenes
Cross-Origin-Embedder-Policy	No configurada	Ausente (Low)	Permite que el documento cargue recursos cross-origin sin CORP explícito
Cross-Origin-Resource-Policy	No configurada	Ausente (Low)	Otros sitios pueden incrustar/leer los recursos del frontend sin restricción

6.3 Mecanismo de Ataque de la Alerta Más Crítica

La alerta de mayor riesgo detectada por ZAP es "Content Security Policy (CSP) Header Not Set" (Medium) sobre el frontend. Mecanismo de explotación: al no existir una cabecera CSP, el navegador no restringe qué scripts puede ejecutar la página. Si un atacante logra inyectar HTML o JavaScript en cualquier punto del frontend (por ejemplo, un campo de perfil, un nombre de producto o un mensaje de chat que se renderice sin escapar), el script inyectado se ejecuta sin ninguna restricción de origen, cabecera o política que lo bloquee, habilitando un ataque de Cross-Site Scripting (XSS) completo: robo de la sesión/JWT almacenado en el navegador, redirección a sitios maliciosos o modificación del DOM para engañar al usuario. La ausencia simultánea de la cabecera anti-clickjacking permite además incrustar el sitio en un iframe controlado por el atacante para ataques de superposición visual (UI redress).

6.4 Hallazgo Identificado por Revisión de Código

Durante la revisión del código fuente (server.ts) se identificó una inconsistencia de seguridad que ZAP no detecta porque afecta al protocolo WebSocket y no al HTTP tradicional:

Campo	Detalle
Hallazgo	CORS Wildcard en Socket.IO
Severidad	Medium
Archivo	server.ts, línea ~10
Código vulnerable	<code>const io = new Server(server, { cors: { origin: "*" } })</code>
Descripción	El servidor Socket.IO acepta conexiones WebSocket desde cualquier origen, mientras que la API REST tiene CORS configurado solo para orígenes autorizados. Esta inconsistencia permite Cross-Site WebSocket Hijacking (CSWSH).
Cabecera afectada	Access-Control-Allow-Origin
OWASP	A05:2021 Security Misconfiguration
Mitigación	Cambiar <code>origin: "*" por origin: allowedOrigins</code> , el mismo array usado en la API REST

7. SCA Análisis de Dependencias

Se ejecutó npm audit sobre el directorio raíz de cada repositorio (backend y frontend) para identificar vulnerabilidades conocidas (CVEs/advisories) en las dependencias de terceros declaradas en cada package.json.

Comando ejecutado: npm audit (en TuCampus_Backend y TuCampus_Frontend)

Salida real de npm audit — Backend (TuCampus_Backend, rama modelomat):

```
$ npm audit
# npm audit report

nodemailer <=9.0.0
Severity: high
Nodemailer: Message-level raw option bypasses disableFileAccess/disableUrlAccess, enabling arbitrary file read and full-response SSRF in the delivered message - https://github.com/advisories/GHSA-p6gq-j5cr-w38f
fix available via `npm audit fix --force`
Will install nodemailer@9.0.5, which is a breaking change
node_modules/nodemailer

uuid <11.1.1
Severity: moderate
uuid: Missing buffer bounds check in v3/v5/v6 when buf is provided - https://github.com/advisories/GHSA-w5hq-g745-h8pq
fix available via `npm audit fix --force`
Will install uuid@11.1.2, which is a breaking change
node_modules/uuid
  mercadopago 1.0.0 - 3.0.0
    Depends on vulnerable versions of uuid
    node_modules/mercadopago

3 vulnerabilities (2 moderate, 1 high)

To address all issues (including breaking changes), run:
npm audit fix --force
```

Salida real de npm audit — Frontend (TuCampus_Frontend, modelomat)

```
$ npm audit
# npm audit report

uuid <11.1.1
Severity: moderate
uuid: Missing buffer bounds check in v3/v5/v6 when buf is provided - https://github.com/advisories/GHSA-w5hq-g745-h8pq
fix available via `npm audit fix --force`
Will install uuid@11.1.2, which is a breaking change
node_modules/uuid
  mercadopago 1.0.0 - 3.0.0
    Depends on vulnerable versions of uuid
    node_modules/mercadopago

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
npm audit fix --force
```

7.1 Resumen de Vulnerabilidades

Severidad	Backend	Frontend
High	1	0
Moderate	2	2
Low	0	0
Total	3	2

7.2 Dependencias Vulnerables Documentadas

Paquete	Proyecto	Severidad	CVE / Advisory	Versión corregida
nodemailer ≤ 9.0.0	Backend	High	GHSA-p6gq-j5cr-w38f — la opción raw a nivel de mensaje permite eludir disableFileAccess/disableUrlAccess, habilitando lectura arbitraria de archivos y SSRF	nodemailer@9.0.5 (breaking change)
uuid < 11.1.1	Backend y Frontend	Moderate	GHSA-w5hq-g745-h8pq — falta de verificación de límites del buffer en v3/v5/v6 cuando se provee buf. Arrastrada como dependencia transitiva de mercadopago	mercadopago@3.3.0 (breaking change)

7.3 Justificación Técnica de Dependencias no Actualizadas

Los dos hallazgos actuales requieren cambios que rompen la compatibilidad (breaking changes) y por eso npm audit no los corrige automáticamente sin --force. nodemailer@9.0.5 puede requerir ajustes en la configuración del transportador SendGrid antes de actualizar. El paquete uuid está ligado a mercadopago tanto en el backend como en el frontend, por lo que su actualización requiere validar primero la compatibilidad con la integración de pagos (mercadopago@3.3.0) antes de aplicar npm audit fix --force en ambos repositorios. Ninguna dependencia se actualizó todavía en esta entrega; queda documentada como pendiente con la justificación anterior (ver 9.1 requisito SA.5).

8. Configuración Segura y Hardening

8.1 Cabeceras HTTP de Seguridad

El backend implementa el middleware `helmet()` de manera global en el archivo `app.ts`, lo que configura automáticamente las cabeceras de seguridad recomendadas por OWASP. Las verificaciones mediante herramientas de análisis dinámico (ZAP) y pruebas manuales con `curl` confirman que todas las cabeceras críticas se encuentran presentes y operativas en este componente.

Por el contrario, el frontend no cuenta con un mecanismo equivalente de forma nativa en su etapa de desarrollo actual, ya que es servido directamente mediante Vite. El análisis con OWASP ZAP evidenció la ausencia de cabeceras fundamentales como Content-Security-Policy (CSP), X-Frame-Options, X-Content-Type-Options, Permissions-Policy y las directivas Cross-Origin-*. Para mitigar este riesgo de cara al despliegue, se recomienda servir el *build* estático del frontend a través de un proxy inverso (como Nginx, Vercel o Netlify) que permita inyectar estas cabeceras de seguridad, o bien incorporar un complemento de Vite equivalente.

Cabecera	Estado	Método de Implementación
Content-Security-Policy (Backend)	Implementada	helmet() (configuración por defecto)
Strict-Transport-Security (Backend)	Implementada	helmet() (max-age=31536000)
X-Frame-Options (Backend)	Implementada	helmet() (SAMEORIGIN)
X-Content-Type-Options (Backend)	Implementada	helmet() (nosniff)
Content-Security-Policy (Frontend)	Pendiente	No implementada (requiere configuración en el servidor de producción)
X-Frame-Options (Frontend)	Pendiente	No implementada (requiere configuración en el servidor de producción)

8.2 Gestión de Secretos

El proyecto utiliza variables de entorno gestionadas mediante `dotenv` (`.env`). Las credenciales de base de datos, claves JWT, API keys de MercadoPago y SendGrid se almacenan en el archivo `.env` que no debe ser incluido en el repositorio Git. Se verificó que el archivo `.env` está correctamente excluido mediante `.gitignore`.

Secreto	Método de gestión	Estado
Credenciales PostgreSQL	Variable de entorno DATABASE_URL	Correcto
JWT Secret	Variable de entorno JWT_SECRET	Correcto
API Key MercadoPago	Variable de entorno MERCADOPAGO_KEY	Correcto
Credenciales SendGrid	Variable de entorno SENDGRID_KEY	Correcto

8.3 Logging de Eventos de Seguridad

El proyecto utiliza morgan en modo "dev" para logging de peticiones HTTP. Se registran automáticamente todos los códigos de respuesta, incluyendo errores 4xx (no autorizado, no encontrado) y 5xx (errores internos). El sistema de autenticación registra intentos fallidos de login a través del rate limiter authLimiter.

8.4 Configuración de Defaults Seguros

Elemento	Configuración	Estado
Credenciales por defecto	No existen cuentas con contraseñas por defecto	Correcto
Rate limiting global	100 req / 15 min por IP	Implementado
Rate limiting auth	10 intentos / 15 min por IP	Implementado
Validación de archivos	Solo JPEG, PNG, WEBP; máximo 5MB	Implementado
CORS API REST	Solo orígenes autorizados en allowedOrigins	Implementado
CORS WebSocket	origin: "*" ABIERTO	Pendiente corrección

8.5 Checklist de Defensa en Profundidad

Capa	Control implementado	Estado
1. Validación de entrada	validateRegister, validateLogin, validateForgotPassword en middlewares	Implementado
2. Autenticación	JWT con verifyToken middleware en rutas protegidas	Implementado
3. Gestión de sesión	Tokens JWT con expiración, endpoint /api/auth/logout	Implementado
4. Cifrado	HTTPS en producción (HSTS configurado), bcrypt para contraseñas	Implementado
5. Logging	morgan dev + logs de errores en controladores	Parcial

9. Tabla de Trazabilidad de Hallazgos

Hallazgo	Herramienta	OWASP Top 10	CWE	Severidad
parseInt global (Backend, iaController.ts:L86)	SonarQube	A03 Injection	CWE-676	Medium
Import crypto legacy (Backend, authController.ts:L5)	SonarQube	A02 Cryptographic Failures	CWE-477	Medium
Excepciones capturadas sin registrar (Backend y Frontend, 26 ocurrencias)	SonarQube	A09 Security Logging and Monitoring Failures	CWE-390	Medium
CORS abierto en Socket.IO (Backend, server.ts)	Revisión manual	A05 Security Misconfiguration	CWE-942	Medium
CSP Header no configurada (Frontend)	OWASP ZAP	A05 Security Misconfiguration	CWE-693	Medium
Cabecera anti-clickjacking ausente (Frontend)	OWASP ZAP	A05 Security Misconfiguration	CWE-1021	Medium
Subresource Integrity (SRI) ausente (Frontend)	OWASP ZAP	A08 Software and Data Integrity Failures	CWE-353	Medium
nodemailer ≤ 9.0.0 (Backend)	npm audit	A10 SSRF / A06 Vulnerable Components	CWE-918	High
uuid < 11.1.1 (Backend y Frontend, vía mercadopago)	npm audit	A06 Vulnerable Components	CWE-125	Moderate
parseInt global (Backend, iaController.ts:L86)	SonarQube	A03 Injection	CWE-676	Medium

10. Dominio Conceptual Aplicado

10.1 SAST, DAST, IAST y SCA

Las cuatro metodologías son complementarias: SAST encuentra problemas en el código sin ejecutarlo, DAST encuentra problemas de comportamiento en tiempo de ejecución, IAST combina ambas perspectivas con un agente interno, y SCA identifica vulnerabilidades en librerías de terceros que ninguna de las otras tres analiza directamente.

Tipo	Herramienta
SAST	Analiza el código fuente sin ejecutarlo. Usa taint analysis y pattern matching. Herramientas: SonarQube, Semgrep. Se usa en desarrollo temprano.
DAST	Analiza la aplicación en ejecución desde afuera mediante fuzzing y active scanning. Se usa en staging. Herramienta: OWASP ZAP.
IAST	Combina agente interno con tráfico real. No se usó en esta auditoría por requerir instrumentación especial del runtime.
SCA	Analiza dependencias de terceros buscando CVEs conocidos. Se usa en cualquier etapa. Herramienta: npm audit.

10.2 Mecanismo interno de SAST y DAST

SAST utiliza taint analysis para rastrear datos no confiables desde su entrada hasta su uso en operaciones sensibles. También usa pattern matching para detectar patrones inseguros conocidos, como el uso de `parseInt` sin `Number.parseInt` detectado por SonarQube en `iaController.ts` (L86).

DAST utiliza fuzzing para enviar entradas maliciosas automatizadas a los endpoints y observar respuestas anómalas. También realiza active scanning probando configuraciones inseguras como cabeceras HTTP ausentes. OWASP ZAP ejecutó este análisis contra el backend (<http://192.168.0.4:3000>), confirmando que helmet configura correctamente todas las cabeceras de seguridad, y contra el frontend (<http://192.168.0.4:5173>), donde detectó la ausencia de CSP y otras cabeceras al no existir un middleware de seguridad equivalente a helmet.

10.3 Las 6 categorías STRIDE con ejemplo propio

Categoría STRIDE	Ejemplo en TuCampus
Spoofing	Suplantar identidad. Ejemplo: un atacante roba el JWT de un alumno mediante XSS y realiza pedidos en su nombre.
Tampering	Modificar datos en tránsito. Ejemplo: interceptar POST /api/orders y cambiar el campo precio a cero antes de llegar al servidor.
Repudiation	Negar una acción realizada. Ejemplo: alumno niega haber hecho un pedido; sin logs firmados no hay evidencia.
Information Disclosure	Exponer información sensible. Ejemplo: CORS abierto en Socket.IO permite leer mensajes del chat desde cualquier origen.
Denial of Service	Hacer el sistema inaccesible. Ejemplo: flood de peticiones a /api/ia con textos largos satura el modelo y bloquea el servidor.
Elevation of Privilege	Obtener permisos mayores. Ejemplo: alumno modifica campo rol en JWT para acceder a /api/admin reservado al Administrador Maestro.

10.4 Shift Left Security y pipeline DevSecOps

Shift Left Security significa mover las pruebas de seguridad lo más temprano posible en el ciclo de desarrollo. Para TuCampus, el pipeline DevSecOps se organizaría: Pre-commit con Semgrep local (antes de subir código), CI con SonarQube + npm audit automático (en cada push), y Staging con OWASP ZAP (antes de cada despliegue a producción). Esto reduce el costo de corrección porque los defectos se encuentran cuando el código aún está fresco.

10.5 Falso positivo vs falso negativo

Un falso positivo ocurre cuando la herramienta reporta una vulnerabilidad que no existe. Ejemplo: SonarQube marca parseInt como inseguro en un contexto donde la entrada ya está validada. El falso positivo genera ruido pero no riesgo directo.

Un falso negativo ocurre cuando la herramienta NO reporta una vulnerabilidad que sí existe. Ejemplo: Semgrep no detectó el CORS abierto en Socket.IO porque no tiene regla específica para esa configuración. El falso negativo es más peligroso porque genera una falsa sensación de seguridad: el equipo cree que el sistema es seguro cuando en realidad contiene vulnerabilidades explotables.

10.6 Obligaciones LFPDPPP para un desarrollador de software

Principio de minimización: recabar solo los datos necesarios. TuCampus recopila nombre, correo, identificación universitaria y fotografía, todos necesarios para el servicio.

Aviso de privacidad: informar al usuario qué datos se recopilan, con qué finalidad y quién los trata, antes de recabarlos.

Medidas de seguridad técnicas: implementar controles que protejan los datos contra acceso no autorizado. TuCampus implementa JWT, helmet, CORS restrictivo y rate limiting.

No transferencia sin consentimiento: los datos no pueden compartirse con terceros sin autorización del titular. La excepción aplicable es MercadoPago como procesador de pagos.

Derechos ARCO: garantizar al titular el derecho de Acceder, Rectificar, Cancelar u Oponerse al tratamiento de sus datos personales mediante un mecanismo accesible en la plataforma.

Anexo A Aviso de Privacidad LFPDPPP

Responsable del tratamiento de datos personales

El equipo de desarrollo del proyecto TuCampus, en el marco de la materia Seguridad en el Desarrollo de Aplicaciones, cuatrimestre Mayo-Agosto 2026.

Datos personales recabados

TuCampus recaba los siguientes datos personales: nombre completo, correo electrónico institucional, número de identificación universitaria, fotografía de perfil, historial de pedidos y preferencias de consumo.

Finalidad del tratamiento

Los datos personales son utilizados exclusivamente para: autenticación e identificación del usuario, procesamiento de pedidos en la cafetería y mercado universitario, asignación y validación de becas alimenticias, y comunicación relacionada con el servicio.

Transferencia de datos

Los datos personales no son transferidos a terceros, salvo a los procesadores de pago (MercadoPago) para la realización de transacciones, quienes cuentan con sus propias políticas de privacidad.

Derechos ARCO

El titular de los datos tiene derecho a Acceder, Rectificar, Cancelar u Oponerse al tratamiento de sus datos personales. Para ejercer estos derechos puede contactar al administrador del sistema a través de los canales oficiales de la institución.

Medidas de seguridad técnicas implementadas

TuCampus implementa medidas técnicas de seguridad que incluyen cifrado de contraseñas con bcrypt, autenticación mediante tokens JWT con expiración, control de acceso basado en roles (RBAC), cabeceras HTTP de seguridad configuradas mediante helmet, y rate limiting diferenciado para endpoints de autenticación.

Cambios al aviso de privacidad

Cualquier modificación a este aviso será notificada a los usuarios a través de la plataforma TuCampus. La versión vigente estará siempre disponible en la sección de privacidad del sistema.

Reporte elaborado por:

Arlet Roxana Herrera Vázquez

Andrea Vasquez Noyola

Guillermo Larzen Vazquez Rangel

Fecha: 10 de agosto de 2026